CS512
Machine Learning

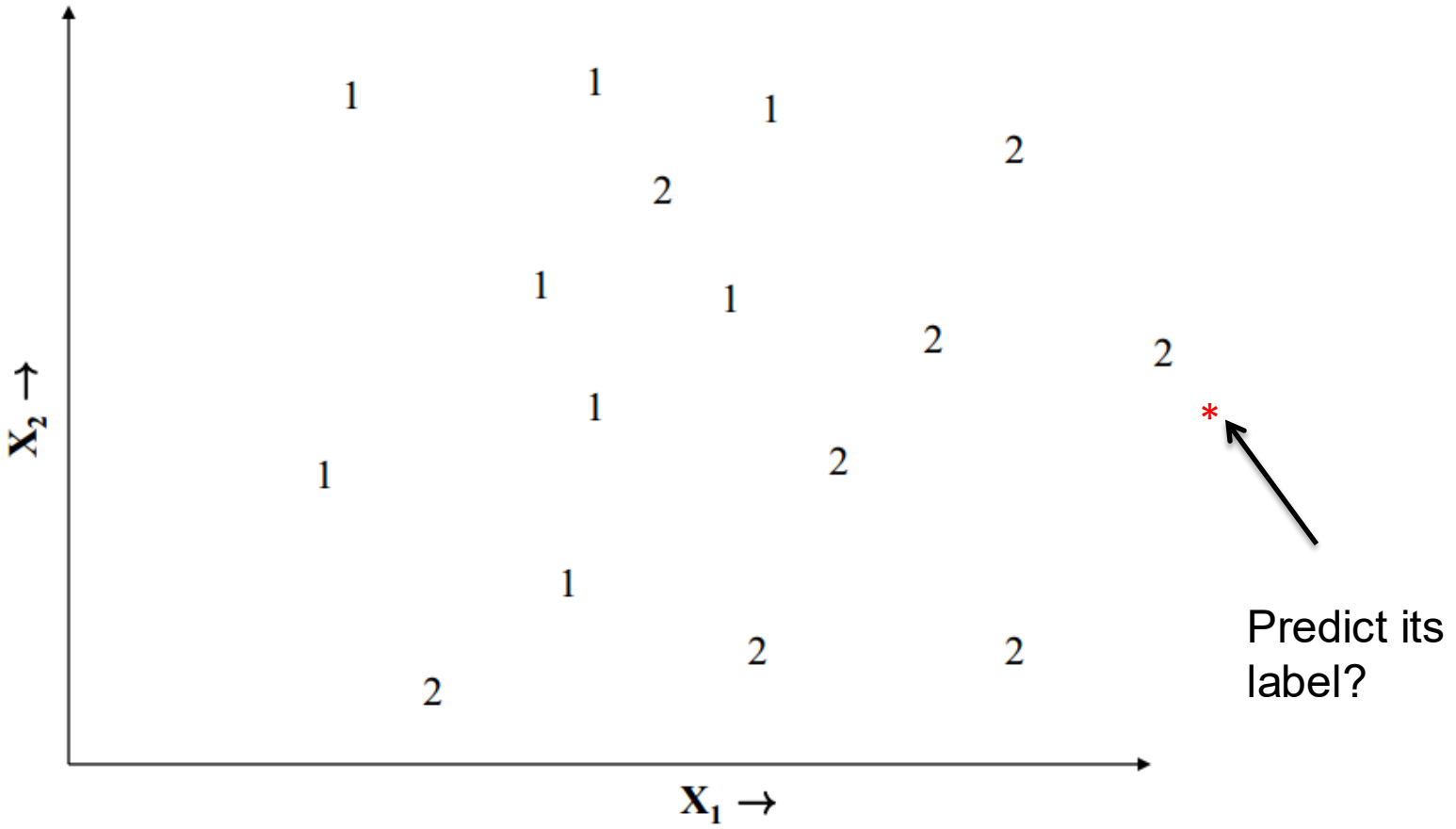
k-Nearest Neighbour

Sabanci University
Öznur Taştan

Outline

- Nearest Neighbor Classifier
- k-Nearest Neighbor for Classification and Regression
- Important Decisions
  - Value of k
  - Distance Measure
  - Voting Mechanism
- Curse of Dimensionality
- Other Practical Issues

How would you classify ?



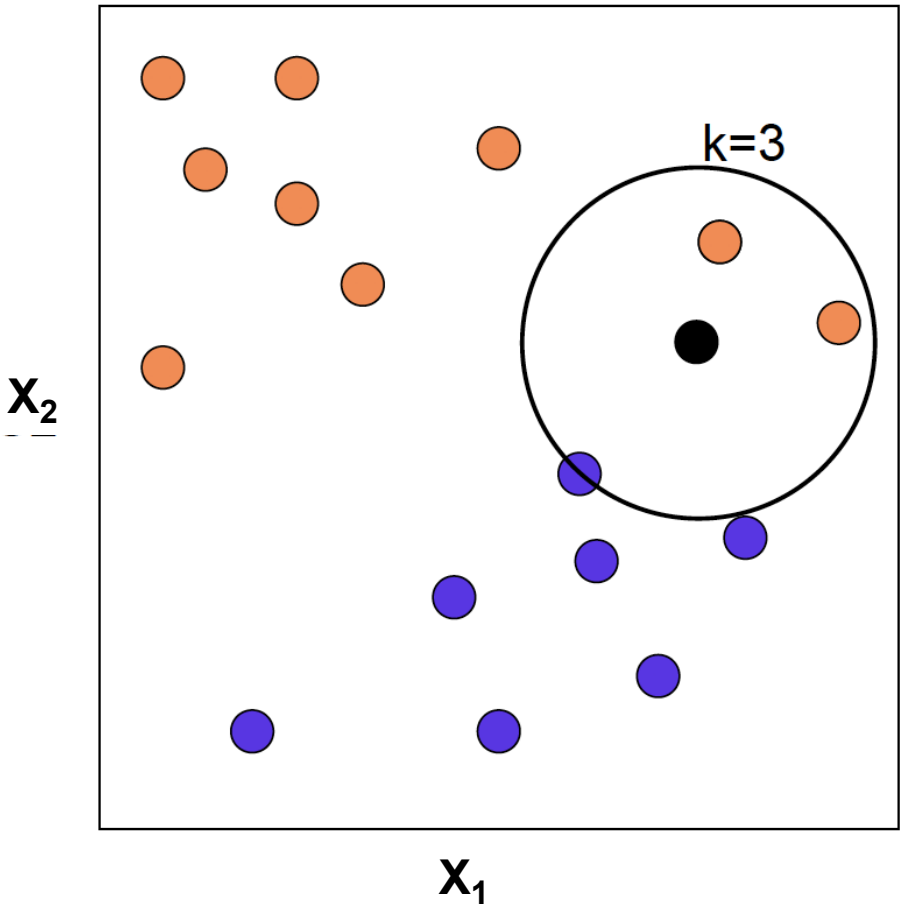
1- Nearest Neighbor Classifier Algorithm

- **Learning step:**
  - Store training examples

- **Prediction step:**
  - To classify a new example find the closest example to the test example

  - Predict the class with that example label.

K-Nearest Neighbour(kNN)



kNN Classifier

- Assumes all instances are points in d-dimensional space
- Classification
  - Find its k nearest neighbors & pick the majority class label of these
- Regression
  - Find its k nearest neighbors & calculate the average value of these
- Closeness depends on the distance measure
- Need to specify both the distance measure and k in advance

k-NN for Regression

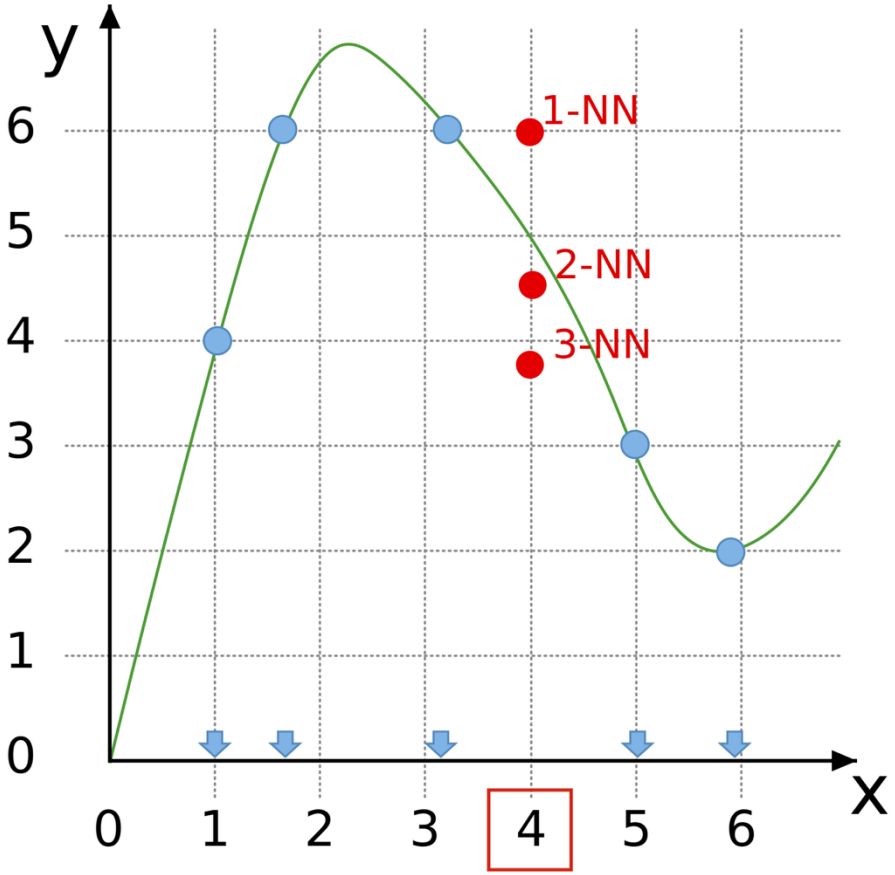


Image credit: Victor Lavrenko

Nearest Neigbhour Algorithm

Algorithm (1-NN):

1. **Given:** training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{1, \ldots, C\}$; query point $\mathbf{x}$.
2. **Find the nearest neighbor** of $\mathbf{x}$ in the training set under a chosen distance $D(\cdot, \cdot)$:

$$i^* = \arg \min_{i \in \{1,\ldots,N\}} D(\mathbf{x}^{(i)}, \mathbf{x}).$$

3. **Predict:**

$$\hat{y} = y^{(i^*)}.$$

4. **Output** $y = \hat{y}$.

Note: The 1-NN rule assigns to $\mathbf{x}$ the label of the single closest training example.

K-NN algorithm

Algorithm (k-NN):

1. **Given:** training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{1, \ldots, C\}$; query point $\mathbf{x}$; integer $k \geq 1$.

2. **Find the k nearest neighbors** of $\mathbf{x}$ in the training set under a chosen distance $D(\cdot, \cdot)$:

   $\mathcal{N}_k(\mathbf{x}) =$ indices of the k training points with smallest $D(\mathbf{x}^{(i)}, \mathbf{x})$.

3. **Predict (classification) by majority vote:**

$$\hat{y} = \arg \max_{c \in \{1,\ldots,C\}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}\left[y^{(i)} = c\right].$$

4. **Output** $y = \hat{y}$.

Probabilistic View

Derive a distribution over the outputs for the local region around ***x***

$$\hat{P}(y = c \mid \mathbf{x}, \mathcal{D}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}, \mathcal{D})} \mathbf{1}[y^{(i)} = c]$$

k-nearest neighbours of ***x*** in dataset *D*

K-NN

- Instead of devising one **global model or approximation of the target function**, for each different data point, there is a different **local approximation.**

- This local approximation depends on the data point itself as well as the training data points.

1-NN Decision Boundary

- Take union of the pair-wise decision boundaries of instances of the same class
- Voronoi diagram

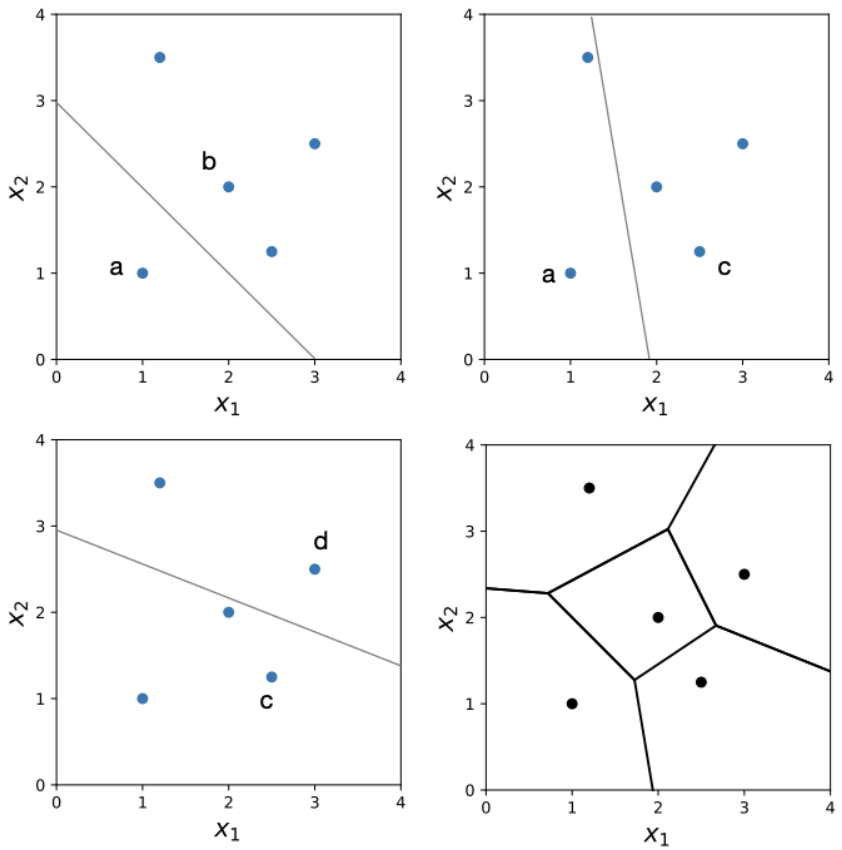
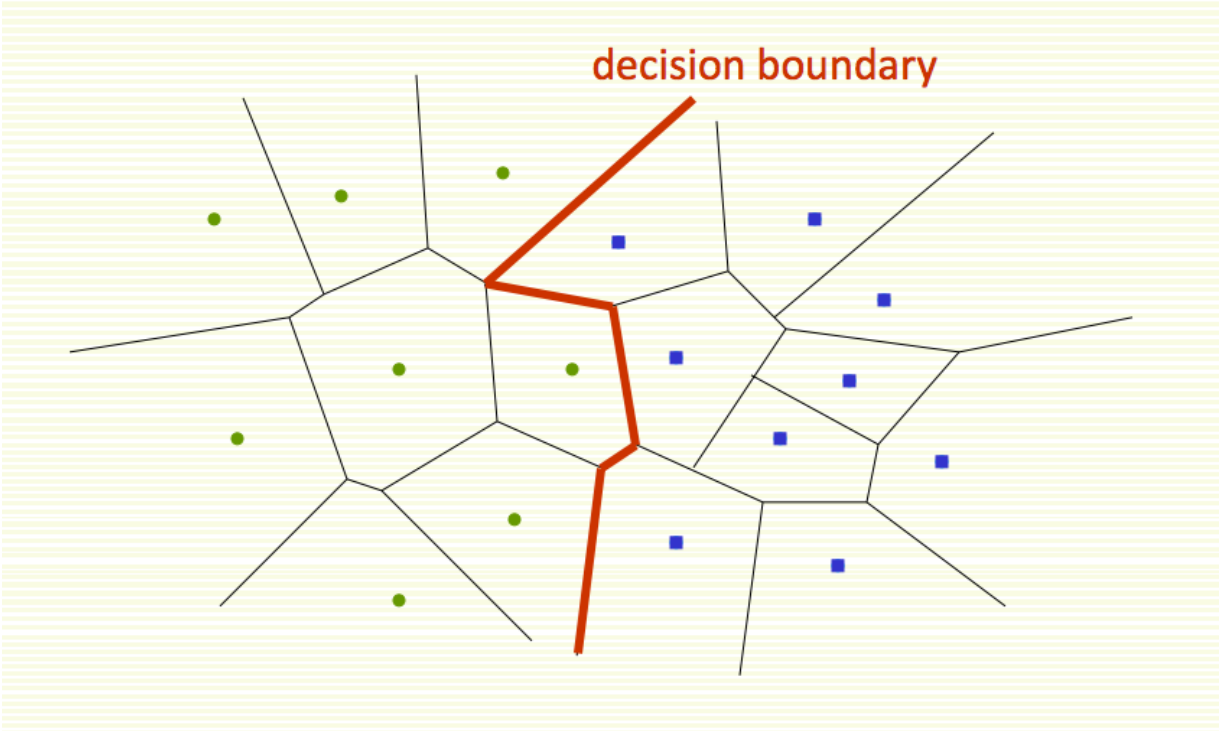


Image Credit: Sebastian Raschka

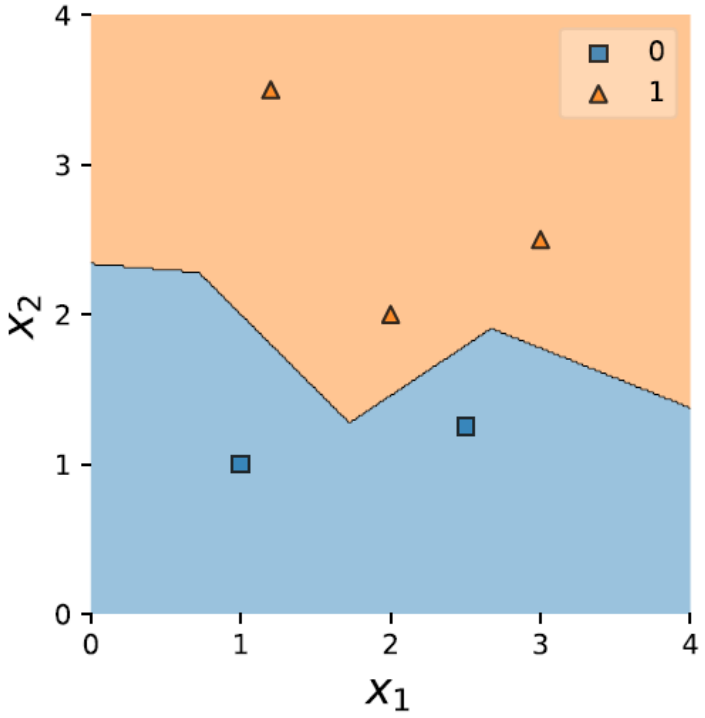
1- NN Decision Boundary

- The nearest neighbor algorithm does not explicitly compute decision boundaries.
- The decision boundaries form a subset of the Voronoi diagram for the training data.



1-NN Decision Boundary

- Decision boundary of 1-NN (it is a piecewise linear decision boundary, it is a nonlinear decision boudnary at the end of the day)

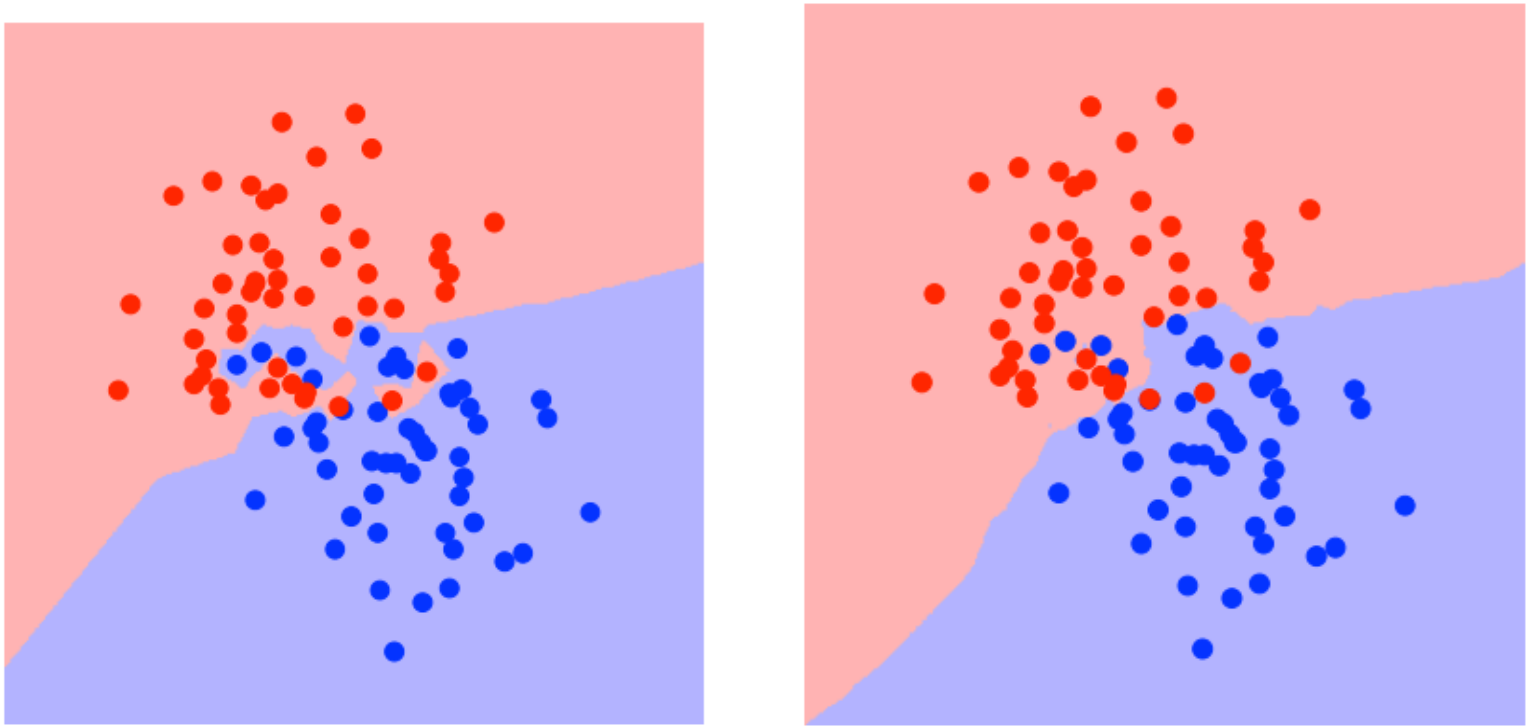


Important Decisions

- Value of k
- Distance measure
- Voting mechanism

k-NN Decision Boundary

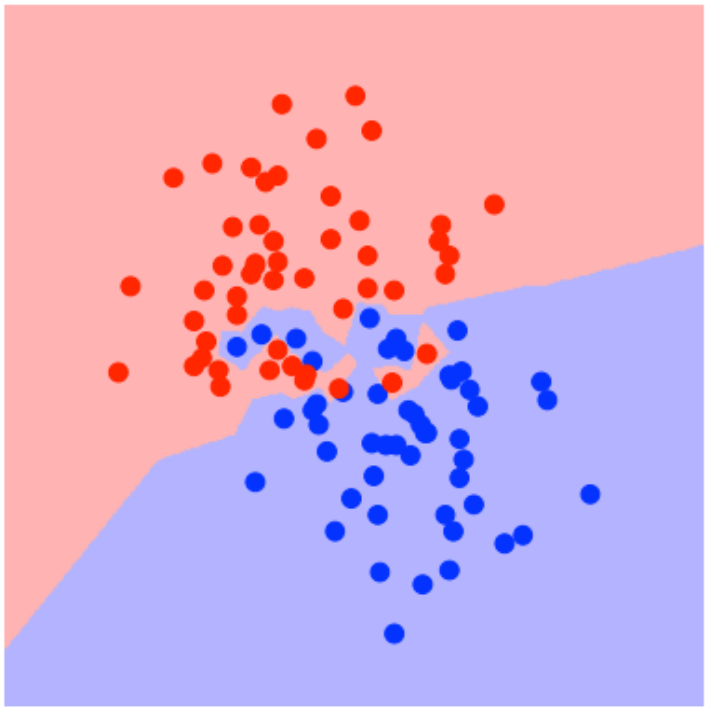
- How does the decision boundary change with k? Which ones is k=1? Which one is k=7?



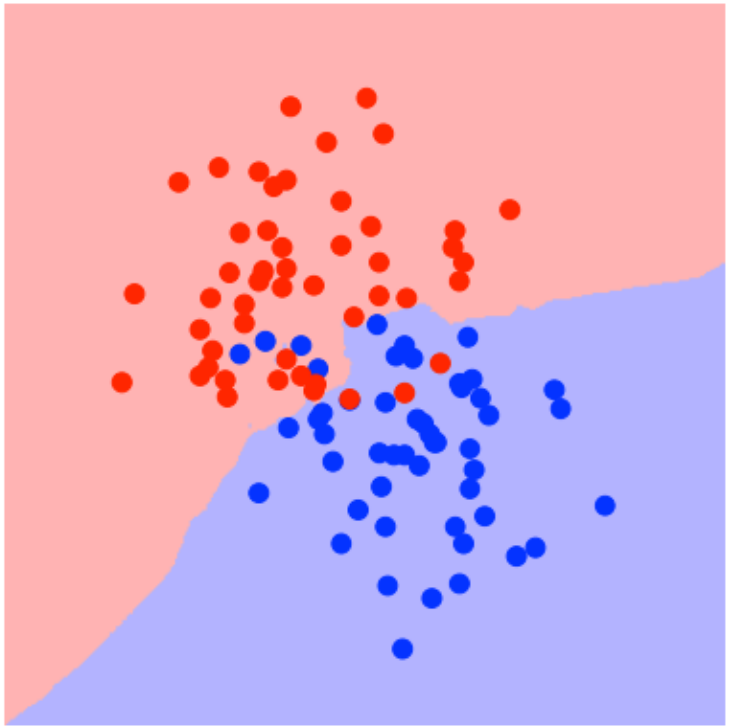
k-NN Decision Boundary

- How does the decision boundary change with k? Which ones is k=1? Which one is k=7?

k=1

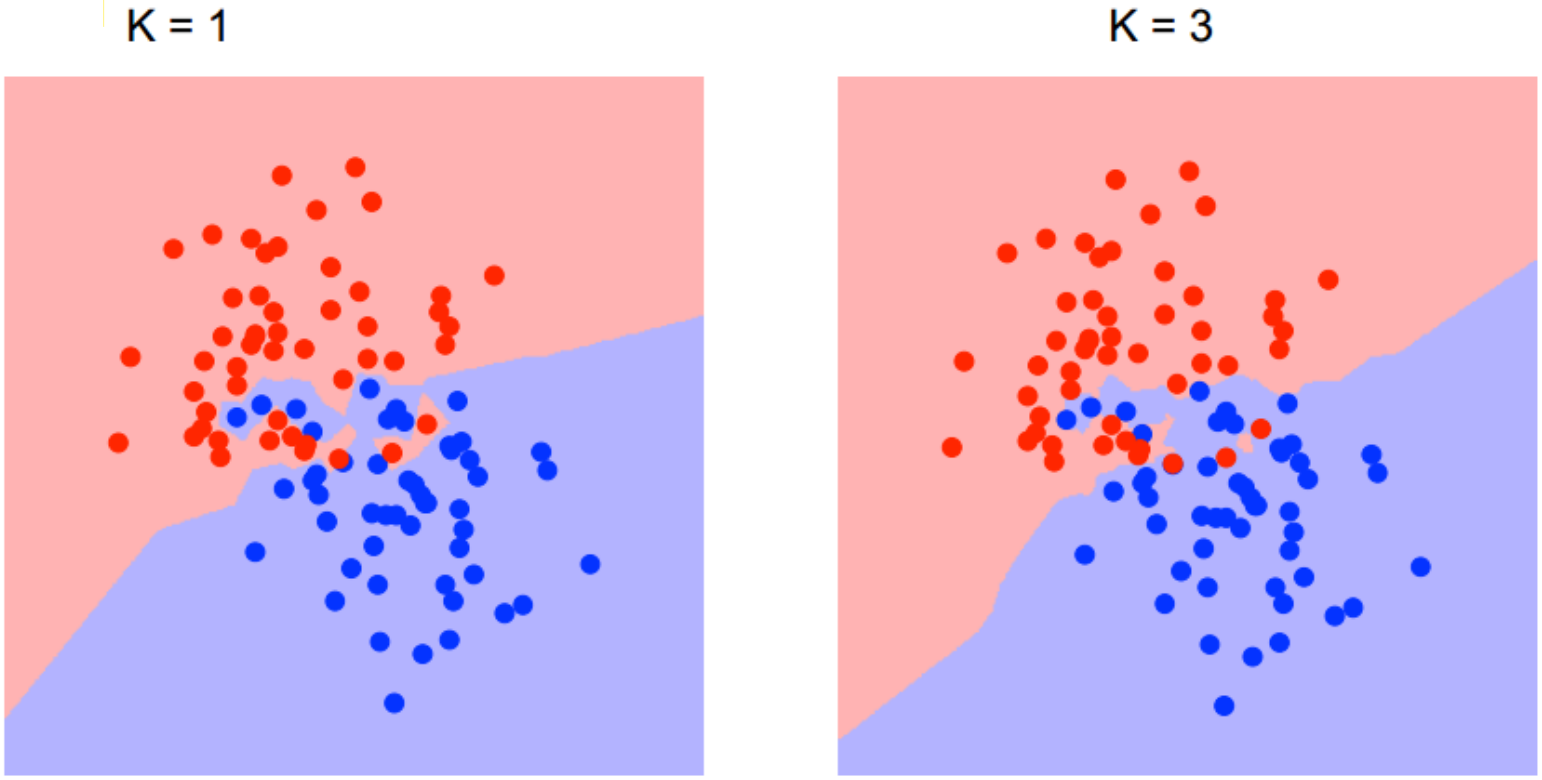


k=7



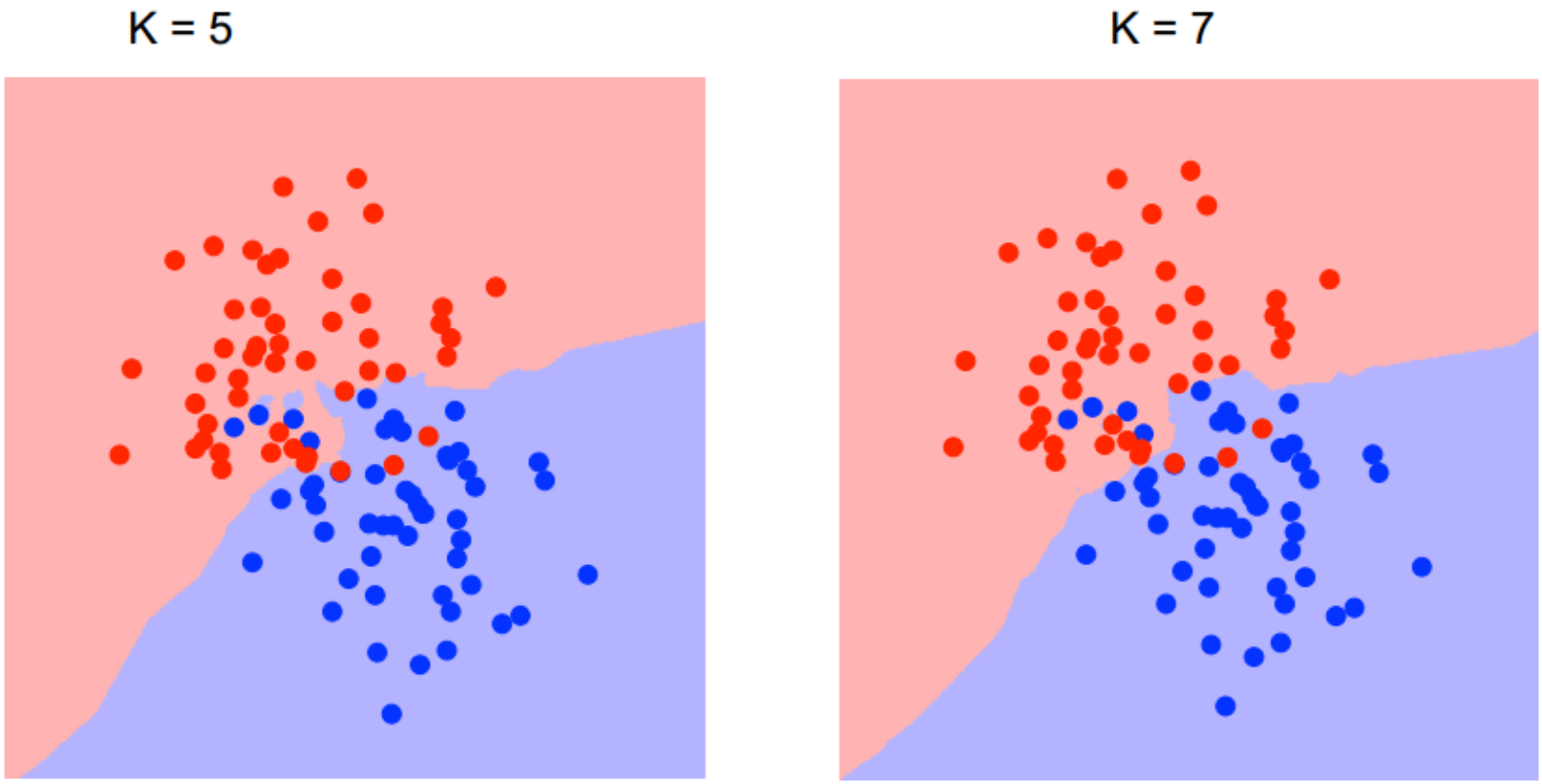
k-NN Decision Boundary

- Piecewise linear decision boundary
- Increasing *k* simplifies the decision boundary
  - Majority voting means less emphasis on individual points



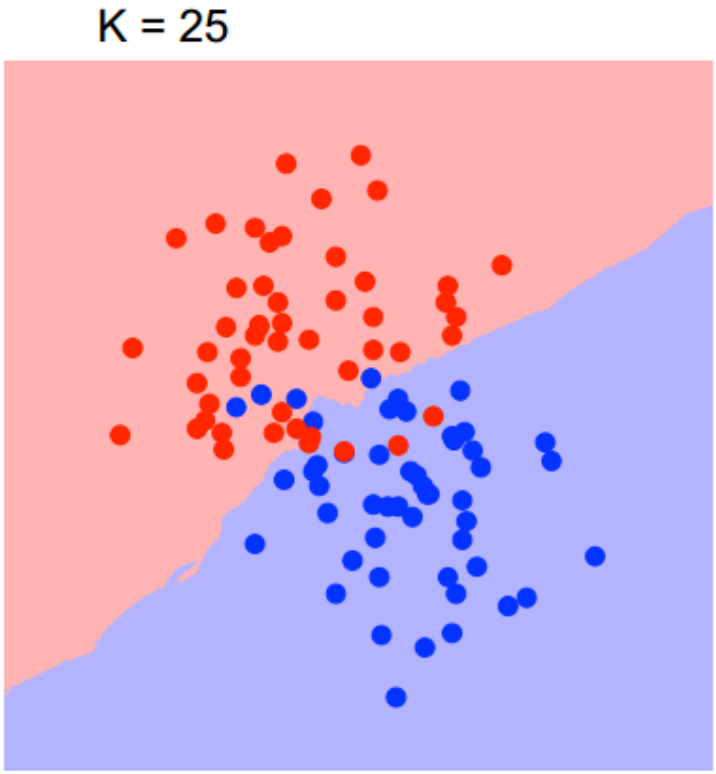
k-NN Decision Boundary

- Piecewise linear decision boundary
- Increasing *k* simplifies the decision boundary
  - Majority voting means less emphasis on individual points



k-NN Decision Boundary

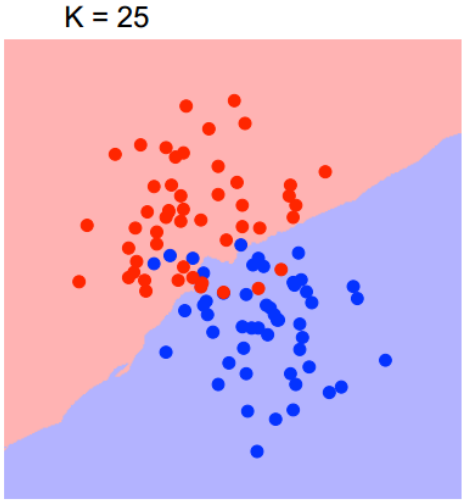
- Piecewise linear decision boundary
- Increasing *k* simplifies the decision boundary
  - Majority voting means less emphasis on individual points



Effect of k

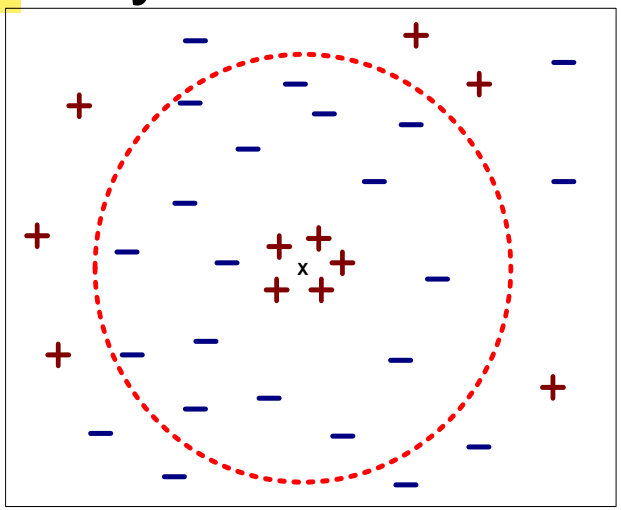
- Higher *k* yields smoother predictions, since we average over more data

- But could also be too smooth. Think of the extreme case k = N , where N is number of training examples.

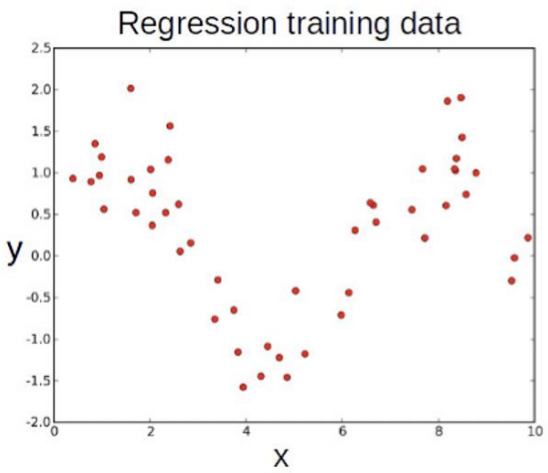


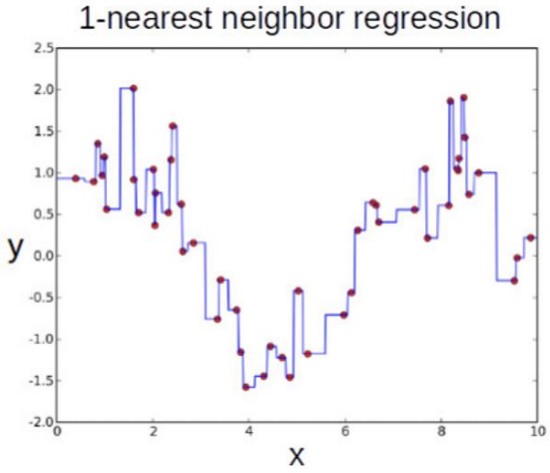
Effect of k

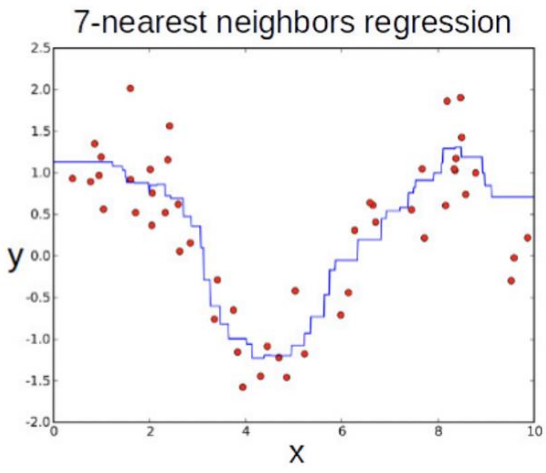
- Large value of k
  - Everything is classified as the most probable class
- Small value of k
  - Highly variable, small change to the training data will lead to large changes in the classifier
- Affects smoothness of the boundary



Affect of k on k-NN Regression







Red Dots: The actual training data points. Each dot has an X value (its position horizontally) and a y value (its position vertically). Blue Line: The model's predictions ($\hat{y}$). For any given X value on the horizontal axis, the blue line shows what value of y the k-NN model estimates based on its neighboring training points.

Selecting k

- How to select k?
  - On your validation data use different values of k and observe the error
  - Pick k that gives the best generalization performance

Important Decisions

- Value of k
- Distance measure
- Voting mechanism

Distance Measures

- Key component in k-NN algorithm
- Defines which examples are similar and which are not
- Can have strong effect on performance
- There is no best distance measure
- The choice depends on the context or task

Distance

- A numerical measure of how different (dissimilar) two data objects are
- Distance measure
  - A function which maps pairs of objects to real values
  - Lower when object are more similar
  - Minimum value is 0 (when object is compared against itself)
  - Maximum value can vary

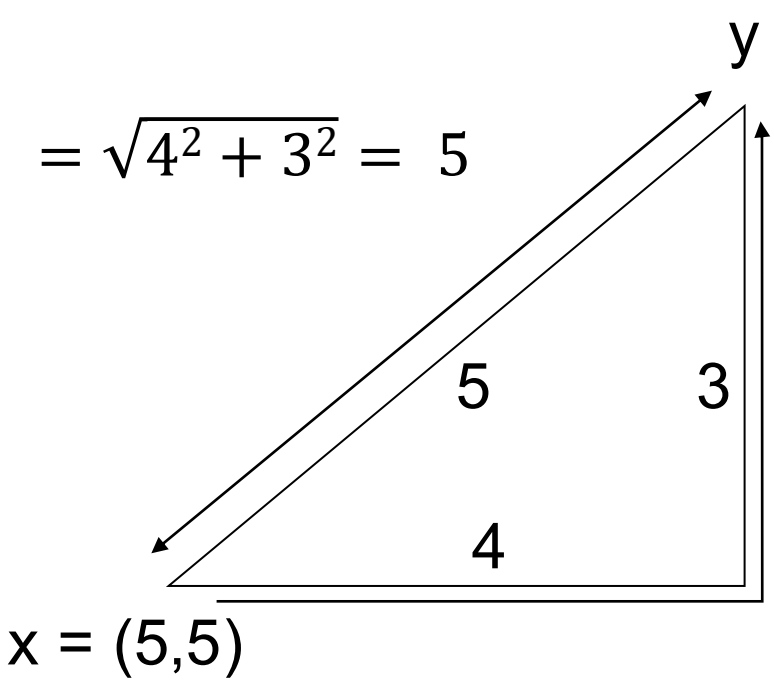
Distance Metric

- A distance function d is a **distance metric** if it is a function from pairs of objects to real numbers such that:
  - $d(x,y) \geq 0$ (non-negativity)
  - $d(x,y) = 0$ iff $x = y$ (identity)
  - $d(x,y) = d(y,x)$ (symmetry)
  - $d(x,y) \leq d(x,z) + d(z,y)$ (triangle inequality)

Example of Distances

L_2-norm:

$$dist(x, y) = \sqrt{4^2 + 3^2} = 5$$



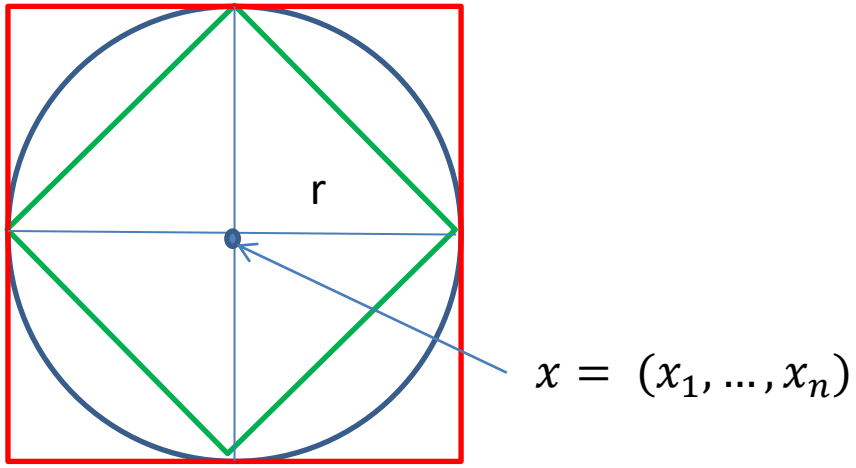
L_1-norm:

$$dist(x, y) = 4 + 3 = 7$$

L_∞-norm:

$$dist(x, y) = \max\{3,4\} = 4$$

Example



Green: All points y at distance $L_1(x,y) = r$ from point x

Blue: All points y at distance $L_2(x,y) = r$ from point x

Red: All points y at distance $L_\infty(x,y) = r$ from point x

Hamming Distance

- **Hamming distance** is the number of positions in which bit-vectors differ.
  - Example: p_1 = 10101
    p_2 = 10011

    $d(p_1, p_2)$ = 2 because the bit-vectors differ in the 3rd and 4th positions.

- **Hamming distance** between two vectors of categorical attributes is the number of positions in which they differ.
  - Example: x = (red, tall, heavy),
    y = (green, tall, light)
    d(x,y) = 2

Distance Between Strings

- How do we define similarity between strings?

weird	wierd
intelligent	unintelligent
Athena	Athina

- Important for recognizing and correcting typing errors and analyzing DNA sequences

Edit Distance for Strings

- The **edit distance** of two strings is the minimum number of inserts and deletes of characters needed to turn one into the other.

- Example: x = abcde ; y = bcduve.
  - Turn *x* into *y* by deleting a, then inserting u and v after d.
  - Edit distance = 3

- Minimum number of operations can be computed using dynamic programming

Distances Between Distributions

- For instance, we can view a document as a distribution over the words

document	Apple	Google	Trump	Election
D1	0.35	0.5	0.1	0.05
D2	0.40	0.4	0.1	0.1
D3	0.05	0.05	0.6	0.3

- **KL-divergence (Kullback-Leibler)** for distributions P,Q

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

- Asymmetric, we can make it symmetric by taking the average of both sides

Things that Can Affect Distance Calculations

- Scaling issues
  - Features may need to be scaled to prevent distance measures from being dominated by one of the attributes

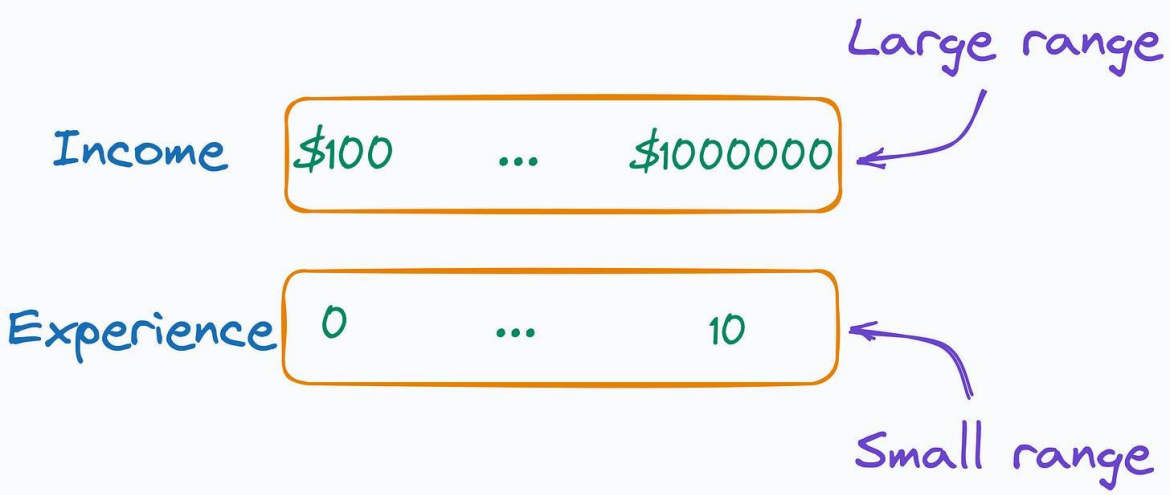


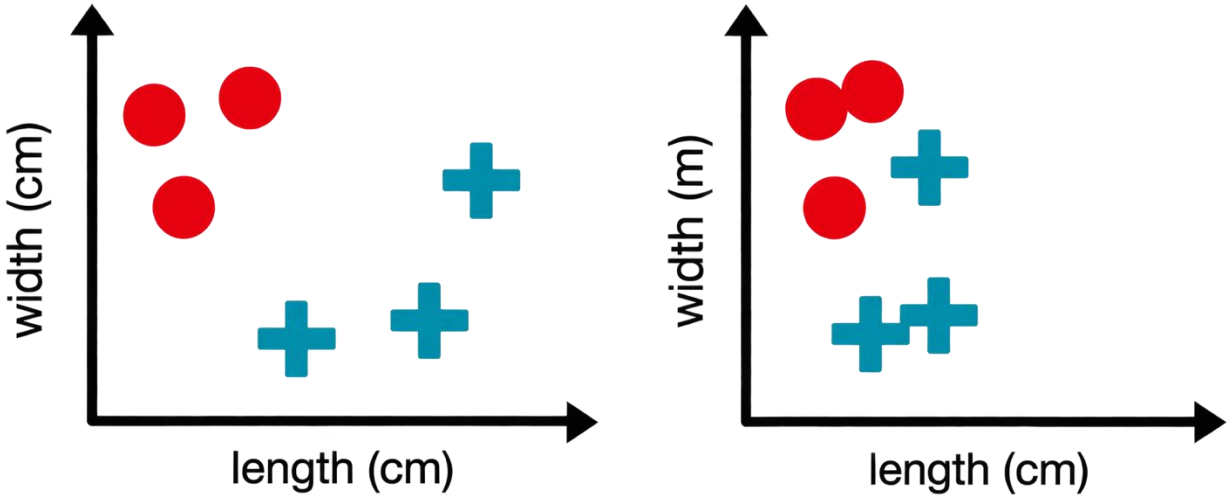
Image credit: https://blog.dailydoseofds.com/p/what-feature-scaling-and-standardization

Euclidean Distance

$$D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt{\sum_{k=1}^{d} \left(x_k^{(i)} - x_k^{(j)} \right)^2}$$

- Treats all dimensions equally
- Sensitive to extreme differences in a single dimension

Scaling Issues Euclidean Distance



Slide adapted from: https://www.cs.cmu.edu/~mgormley/courses/10601-s21/slides/lecture4-knn.pdf

How to scale

Unit range

Scales features to the range [0, 1] (**unit range**).

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

$\min(x)$ and $\max(x)$ are computed per feature across training samples.

Unit variance

Scales features to have zero mean and **unit variance**.

$$x' = \frac{x - \mu}{\sigma}$$

μ and σ are computed per feature across training samples.

Robust scaling

Scales features to have zero median and **unit interquartile range**.

$$x' = \frac{x - \text{median}(x)}{Q_3 - Q_1}$$

$Q_3 - Q_1$ = IQR (interquartile range).
Less sensitive to outliers.

$\text{median}(x)$, Q_3 and Q_1 are computed per feature across training samples.

Missing Values

- Have to “fill in”, unless distance function supports missing values
- Key concern: should affect distance as little as possible

1. Why are Missing Values a Problem?

Distance metrics (like Euclidean or Manhattan distance) require numerical values across all features to compute the exact distance between two points (x and y). If a feature value is missing in either your training data or your query point, the standard distance formula breaks down because you cannot subtract or square a missing value.

---

2. How to Handle Missing Values

You have two main approaches:

* **"Fill In" (Imputation):** You replace the missing values with estimated ones before running the algorithm. Common imputation strategies include:
* Replacing missing values with the **mean** or **median** of that feature across the training set.
* Using advanced imputation methods (like KNN imputation or machine learning models to predict the missing entries).

* **Specialized Distance Functions:** Alternatively, you can use a custom distance metric that is specifically designed to ignore missing dimensions or handle them natively without needing explicit imputation.

---

3. Key Concern

> **"Should affect distance as little as possible"**

* **Why this matters:** When you fill in a missing value, you are essentially inventing data that wasn't originally there. If done poorly, this artificial value will distort the distance calculation, making points look closer or further apart than they actually are.
* **Goal:** Choose an imputation method or distance adjustment that introduces the minimum amount of bias, ensuring your k-NN model's neighborhood decisions remain accurate.

Important Decisions

- Value of k (usually odd)
- Distance measure
- Voting mechanism

Combining of Neighbor Labels

- Options for determining the class from nearest neighbor list
  - Take majority vote of class labels among the *k*-nearest neighbors
  - Weight the votes according to distance
    - example: weight factor $w = 1 / d^2$

Weighted kNN

Algorithm: Weighted k-Nearest Neighbors (Classification)

1. Find the k training examples $(\mathbf{x}^{(i)}, y^{(i)})$ closest to the query point $\mathbf{x}$ using a distance metric (e.g., Euclidean distance):

$$\mathcal{N}_k(\mathbf{x}) = \text{set of } k \text{ nearest neighbors of } \mathbf{x}$$

2. Assign a weight to each neighbor, typically inversely proportional to its distance:

$$w_i = \frac{1}{\text{distance}(\mathbf{x}^{(i)}, \mathbf{x}) + \epsilon}$$

where ϵ is a small constant to avoid division by zero.

3. For each class c, compute the total weight of neighbors belonging to that class:

$$S_c = \sum_{\substack{\mathbf{x}^{(i)} \in \mathcal{N}_k(\mathbf{x}) \\ y^{(i)} = c}} w_i$$

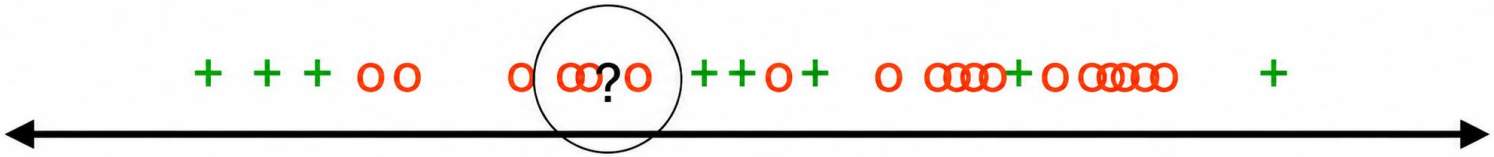
4. Predict the class with the highest total weight:

$$\hat{y} = \arg\max_c S_c$$

Breaking Ties

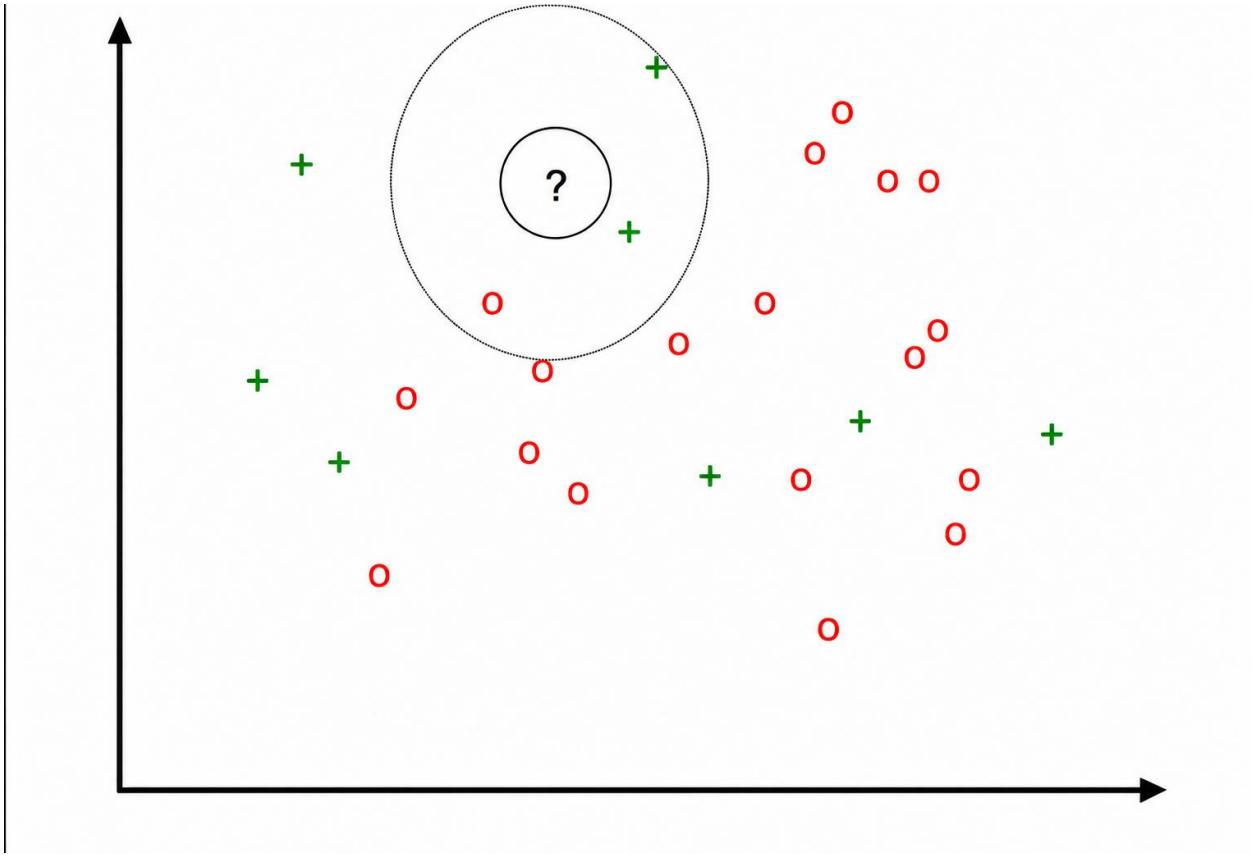
- When you have equal number of different class labeled neighbors, how to resolve ties:
  - Use odd k (does not work for multi-class)
  - Or
    - Decide randomly
    - Pick class with greater prior
    - Use 1-nn classifier to decide

k-NN and Irrelevant features



Based on this single feature, the class label is pretty clear.

k-NN and Irrelevant Features



Adding an irrelevant feature messes it up

More features?

- A toy learning problem:
  - divide the feature space uniformly into bins
  - for each incoming new example
    - figure out the bin the example falls into
    - use the predominant class in that bin as the prediction

Slide credit: Gutierrez-Osuna

More features

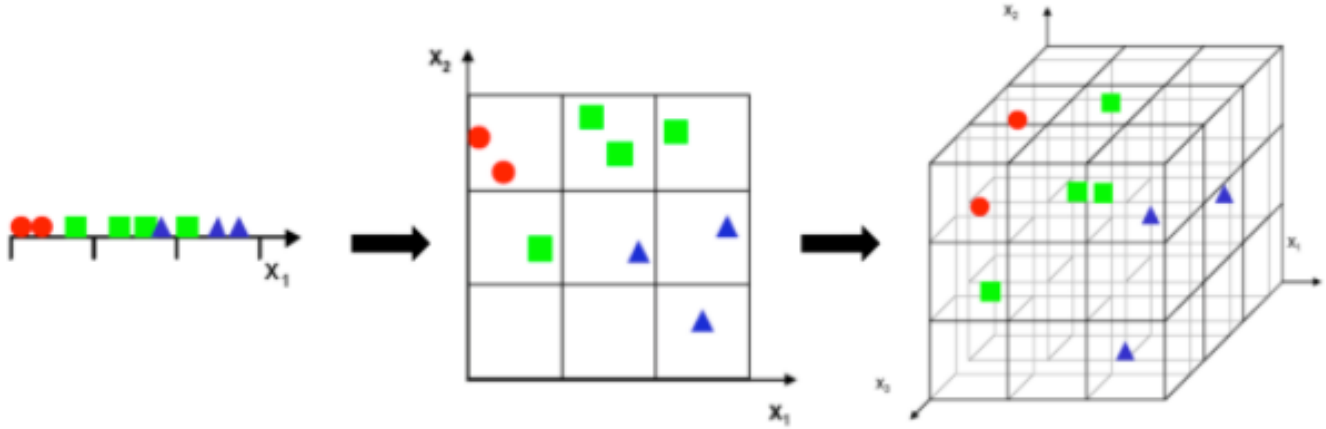
- Example:
  - One feature
  - Divide the space into three bins



  - There is an overlap, so let's add one more features

Slide credit: Reyyan Yeniterzi

More features



With increasing dimensionality,

- # bins required to cover the feature space increases exponentially
- If we want to the same density in each bin (3 examples per bin), we need 81 examples (number of bins $3^3 = 27$ 27x3 =81)
- there won't be enough data to populate each bin
- finding the predominant class in each bin becomes difficult

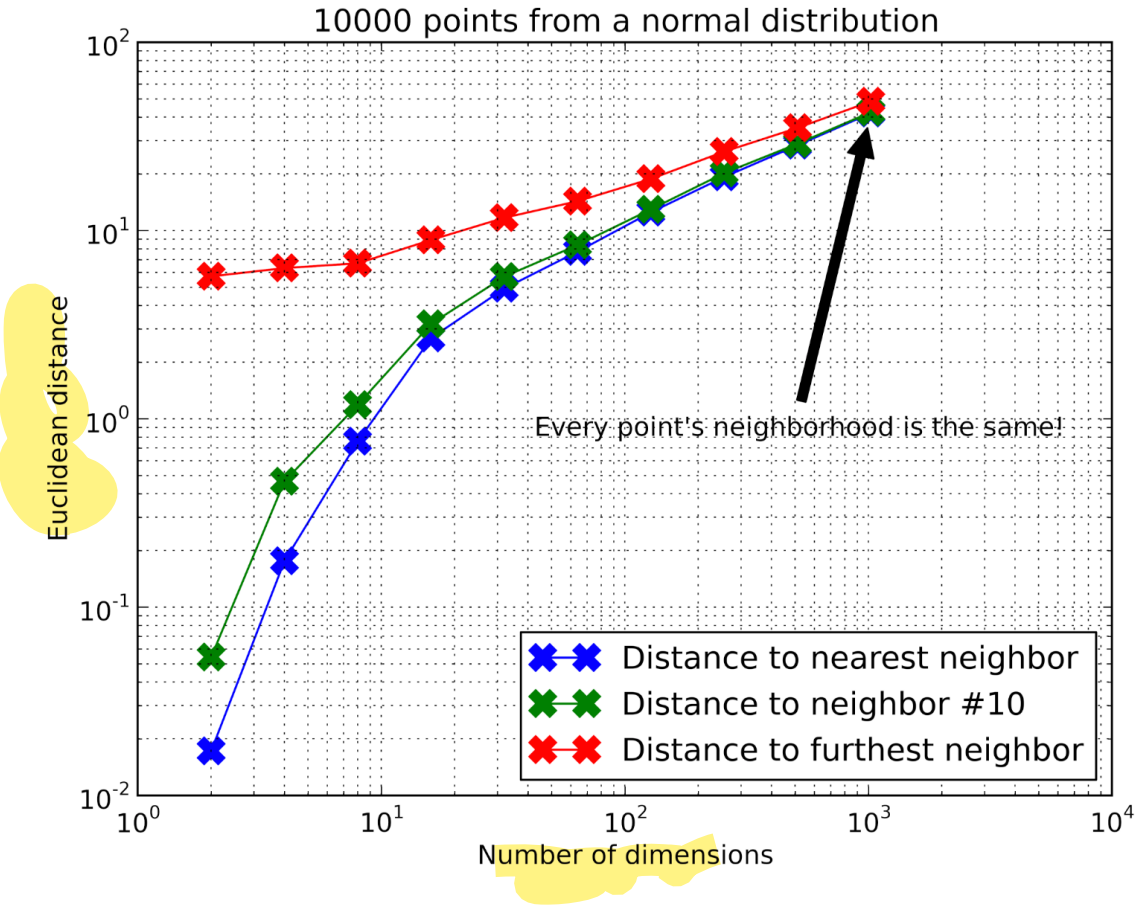
Curse of Dimensionality

- Curse of dimensionality is more of a problem with some machine learning approaches, while not affecting others that much
  - Nearest neighbor approach is most affected

- The problem is that the idea of neighbors gets distorted in an unintuitive way, in (very) high dimensional spaces
  - The sparsity of the data objects in the space is unavoidable
  - It has been shown that in high dimensional space, all pairs of points are almost equidistant from one another for a wide range of data distributions and distance functions.

Curse of Dimensionality

Let's sample a 10,000 points from a normal distribution, then pick a random point in the distribution, and compute the distance to the furthest and closest point:



Source: https://erikbern.com/2015/10/20/nearest-neighbors-and-vector-models-epilogue-curse-of-dimensionality.html

Curse of Dimensionality

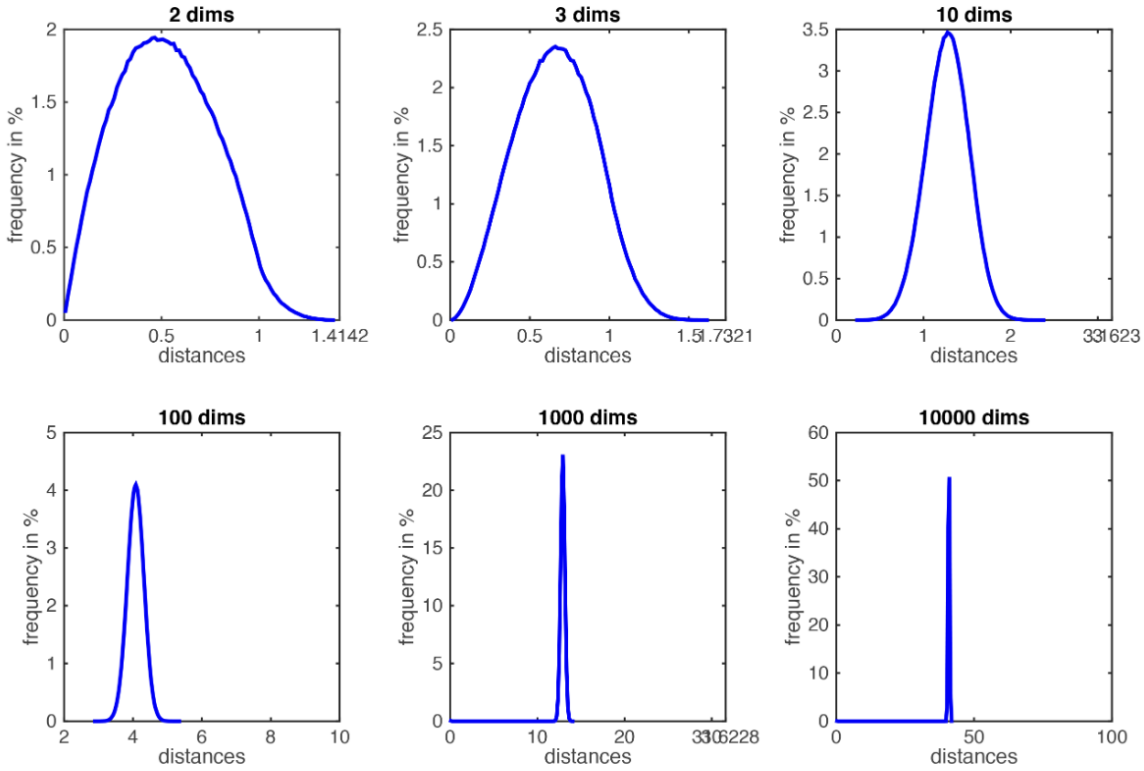


Figure demonstrating ``the curse of dimensionality''. The histogram plots show the distributions of all pairwise distances between randomly distributed points within d-dimensional unit squares. As the number of dimensions d grows, all distances concentrate within a very small range.

Figure source: https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html

k-NN

- K-nearest neighbor is the most basic instance-based
- *k*-Nearest neighbor classifier is a lazy learner
  - Does not build model explicitly defers the work at the test time
  - Classifying unknown samples is relatively expensive
- *k*-Nearest neighbor classifier is a local model, vs. global model of linear classifiers

Issues

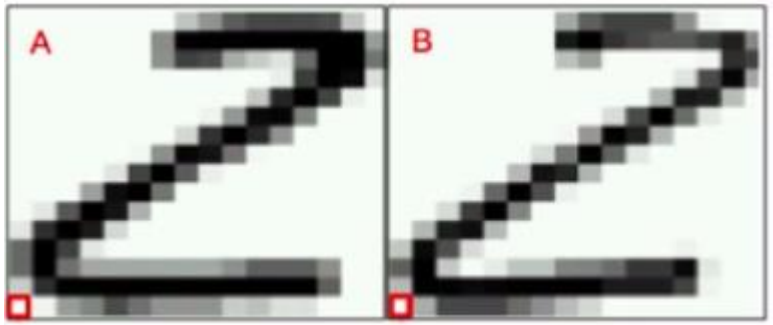
- For high-dimensional space, the problem is that the nearest neighbor **may not be very close at all!**

- **Redundancy** – if you have a bunch of attributes that are essentially redundant, i.e. that are highly correlated to each other, then including them all is tantamount to multiplying the scale of that factor

- Memory-based technique
  - Must make a pass through the data for each classification. This can be prohibitive for large datasets.

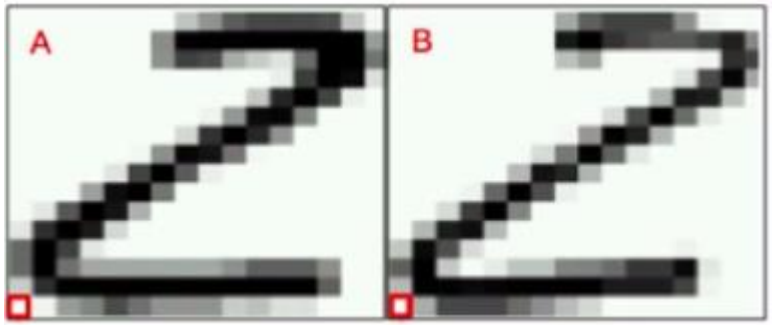
Example: Hand Written Digits

- 16x16 bitmaps
- 8-bit grayscale
- Euclidean distance over pixels

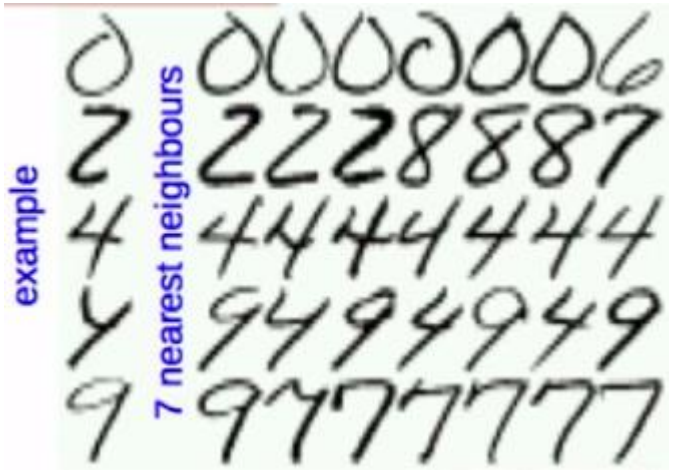


Example: Hand Written Digits

- 16x16 bitmaps
- 8-bit grayscale
- Euclidean distance over pixels

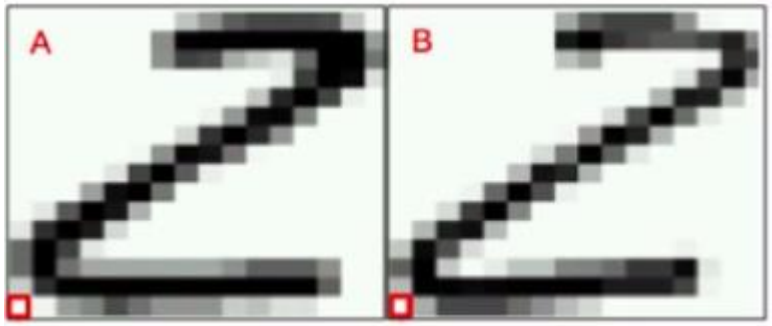


$$D(A,B) = \sqrt{\sum_r \sum_c (A_{r,c} - B_{r,c})^2}$$

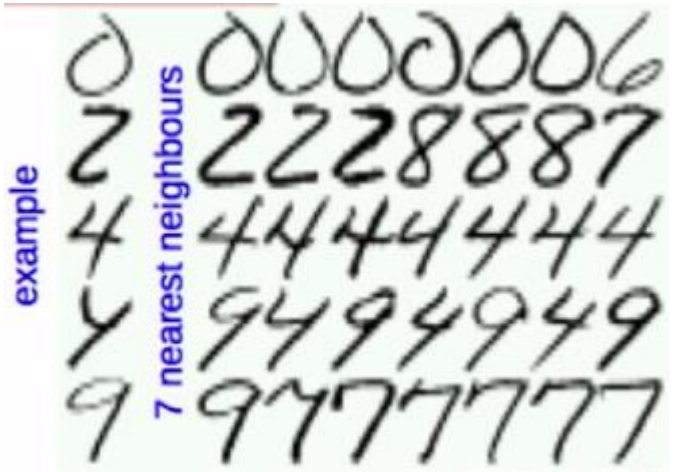


Example: Hand Written Digits

- 16x16 bitmaps
- 8-bit grayscale
- Euclidean distance over pixels



$$D(A,B) = \sqrt{\sum_r \sum_c (A_{r,c} - B_{r,c})^2}$$



Accuracy: 7-NN ~95.2%
SVM ~95.8%
Humans ~97.5%

Improving performance in k-NN

- Try different k values
- Scale the features
- Try different distance measures
- Consider weighted distance measures

Brute-force k-NN: cost per query

- With *N* examples in the training data and *d* features.
- Training stores data, no computation
- Test has to pass through the training data

	Time Complexity	Space Complexity
Training	$\mathcal{O}(1)$	$\mathcal{O}(Nd)$
Testing	$\mathcal{O}(Nd)$	$\mathcal{O}(N)$

This is bad: we want classifiers that are fast at prediction time, training can be slow.

Nearest-neighbor search at scale

- Spotify and Meta use **approximate nearest-neighbor search** to retrieve candidates within their recommendation systems.

- Engineering optimizations make large-scale search practical:
  - **Indexing:** use partitions, graphs, or trees to reduce the number of vectors examined.
  - **Quantization / compression:** reduce memory requirements and distance computation cost.
  - **Parallel / GPU computation:** accelerate distance calculations and neighbor selection.

- **Approximate search** trades some retrieval accuracy for faster queries and, in some methods, lower memory use.

Production grade libraries

An efficient nearest neighbor implementations avoiding brute-force distance checks.

Think of traditional nearest neighbor as looking for your friends in a crowd by checking every face.

Within this analogy, what example efficient k-NN implementations are like:

- **Meta/FAISS:** group people into neighborhoods first, then look inside top neighborhoods.
- **Spotify/Annoy** (Approximate Nearest Neighbors Oh Yeah): divide the city with random cuts and search only relevant regions.

Nearest Neighbor Classifier

- Advantages
  - Simple, with little fitting required.
  - Can model complex, nonlinear decision boundaries.
  - No fixed parametric form.
  - Useful baseline when distance reflects meaningful similarity.
- Disadvantages
  - Prediction can be computationally expensive on large datasets.
  - Requires storing the training examples.
  - Sensitive to feature scaling, irrelevant features, and the distance metric.
  - Suffers from the curse of dimensionality.
  - Performance depends on k, neighbor weighting, and label noise

When to consider k-NN?

- Less than 50 features per instance
  - Dimensionality reduction or feature selection can be used if higher
- Lots of training data
- It is a good baseline to compare against

Acknowledgements

- Alex Smola, Alexander Ihler, Aarthi Singh, Tom Mitchell, Jeff Howbert, Victor Lavrenko, Reyyan Yeniterzi